

Adversarial Attacks and Defenses on Graph-aware Large Language Models (LLMs)

Anonymous submission

Abstract

Large Language Models (LLMs) are increasingly integrated with graph-structured data for tasks like node classification, a domain traditionally dominated by Graph Neural Networks (GNNs). This integration leverages rich relational information to improve task performance, yet the robustness of graph-aware LLMs against adversarial attacks remains unexplored. This paper closes that gap, and makes four contributions. **First**, we present the first systematic study of the adversarial vulnerability of graph-aware LLMs, transferring established graph attack methods for poisoning (training-time) and evasion (test-time) to two representative models, LLAGA (Chen et al. 2024) and GRAPH PROMPTER (Liu et al. 2024). **Second**, we identify a previously unknown attack surface in LLAGA: an attacker can inject malicious nodes as placeholders into the node sequence template to severely degrade its performance. **Third**, we connect these attacks to specific design choices in graph encoding, showing that (1) the node sequence template in LLAGA amplifies its vulnerability; (2) the GNN encoder used in GRAPH PROMPTER is comparatively more robust; and (3) both models remain highly susceptible to imperceptible feature perturbation attacks, an attack vector overlooked in the GNN literature. **Fourth**, building on these findings, we propose GALGUARD, an end-to-end defense framework that pairs an LLM-based feature correction module against feature-level perturbations with adapted GNN defenses against structural attacks. Together, these contributions establish the vulnerabilities of graph-aware LLMs and a first line of defense¹

1 Introduction

Graphs, characterized by nodes and edges that represent entities and relationships, are used to model complex structures in various real-world domains, including social networks (Fan et al. 2019), biology (Dong, Mücke, and Khosla 2022), finance (Wang et al. 2021), and healthcare (Ahmedt-Aristizabal et al. 2021). Graph Neural Networks (GNNs) (Hamilton, Ying, and Leskovec 2017; Kipf and Welling 2022; Veličković et al. 2018), designed specifically for graph-structured data, excel in tasks such as node classification and link prediction. More recently, the success of large language models (LLMs) such as GPT-4 (Achiam et al. 2023), Gemini (Team et al. 2023), LLaMA (Touvron et al. 2023; Dubey et al. 2024), Vicuna (Chiang et al. 2023),

and PaLM-2 (Anil et al. 2023), trained on vast amounts of textual data, has sparked interest in their potential to enhance graph-related tasks. This fusion of graph data with LLMs (called *graph-aware LLMs* (Xie et al. 2023)) has opened a new line of research in which LLMs increasingly perform tasks traditionally handled by GNNs (Ye et al. 2023; Fatemi, Halcrow, and Perozzi 2023; Kuang et al. 2023; Zhang et al. 2023; Chen et al. 2023b, 2024), and recent work shows they can even outperform GNNs on various graph tasks (Wenkel, Wolf, and Knyazev 2023; Chen et al. 2023b; Zhu et al. 2024; Chen et al. 2024), positioning them as powerful alternatives.

As graph-aware LLMs move toward deployment, a basic question has gone unanswered: *are they robust?* Their vulnerabilities to adversarial attacks remain largely unexplored. Existing adversarial attacks on GNNs, spanning poisoning (training-time attacks) and evasion (test-time attacks) (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Dai et al. 2023), operate under a black-box setting in which the attacker has no knowledge of the model’s internal workings. These attacks undermine *model integrity* by subtly altering graph structure or node features to degrade performance. Crucially, while GNN research has focused predominantly on structural attacks, the textual nature of graph-aware LLMs opens a second, largely overlooked front: imperceptible perturbations to node features.

These gaps motivate the central question of this paper: *do adversarial attacks designed for GNNs remain effective against the emerging graph-aware LLM paradigm, and does the way graphs are encoded into the LLM create new vulnerabilities?* Answering this requires accounting for how graphs are integrated into the LLM in the first place. While node features are straightforward for an LLM to process given its proficiency with text, incorporating graph structure is more complex and requires innovative techniques to capture relational information (Chen et al. 2024; Guo et al. 2023; Liu and Wu 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). We therefore organize existing graph-aware methods into two classes by how they encode graphs into LLMs: those that represent graph data as *textual descriptions*, converting graph information into natural language, and those that use *learned projectors* to bridge the gap between the graph and text domains. Since conventional text-based adversarial attacks already suffice to compromise LLMs consuming textual descriptions, we

¹Our code will be released upon acceptance.

focus our investigation on learned projector methods, using two computationally efficient representative models that both build on a frozen pre-trained LLM: LLAGA (Chen et al. 2024) and GRAPH-PROMPTER (Liu et al. 2024).

The central finding of our study is that the very design choices used to encode graphs into LLMs can unexpectedly enlarge the attack surface. Most strikingly, we uncover a new attack surface in LLAGA that lets an attacker severely degrade performance by injecting malicious nodes as placeholders within the node sequence template. We also show that both models are far more vulnerable to imperceptible feature perturbations than to structural ones, and we translate these insights into GALGUARD, an end-to-end graph-aware LLM defense that integrates an LLM-based feature corrector with adapted GNN-based structural defenses. To our knowledge, this is the first study of the adversarial vulnerability of graph-aware LLMs. We summarize our contributions as follows:

- **First robustness study of graph-aware LLMs.** We transfer representative GNN attacks, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), to graph-aware LLMs and quantify the effectiveness of poisoning and evasion attacks against them.
- **Encoding-based taxonomy.** We organize graph-aware LLMs into a taxonomy based on their graph-encoding approach, clarifying how different methodologies integrate graph structure into LLMs and which design choices govern their robustness.
- **A new attack surface.** We discover and characterize a template-injection attack on LLAGA, showing that injecting malicious nodes as placeholders into the node sequence template sharply degrades its performance.
- **Feature-perturbation vulnerability.** We deliver a holistic assessment of graph-aware LLM robustness by launching imperceptible attacks on textual node features, an angle largely ignored in the GNN literature, and show it dominates structural attacks.
- **An end-to-end defense.** We propose GALGUARD, a novel defense that couples LLM-based feature correction with adapted GNN-based structural defenses, and show it consistently restores robustness.

2 Graph Encoding-Based Taxonomy of LLM Adaptations

A central contribution of this paper is a taxonomy that organizes graph-aware LLMs by *how* they encode graphs, because that choice is what determines their attack surface. We identify two main approaches: *textual descriptions of graphs*, where graph information is encoded as natural language, and *learned graph projectors*, where a projector encodes graph data into embeddings or structured forms that LLMs can directly process. We describe each category below; Figure 5 illustrates both. A full description is given in Appendix A.1 and related work on GNN attacks in Appendix A.2.

Graph Representation as a Textual Description. The first family converts graph data into natural language so that

LLMs can process it as textual descriptions, often paired with task-specific query instructions (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a; Zhang 2023). In multimodal applications, Liu et al. (2023) and Su et al. (2022) linked molecule graphs with textual descriptions, and Huang et al. (2023a) studied how to embed graph structure directly into LLM prompts, finding that LLMs tend to treat graph-related prompts as contextual paragraphs – a result that underscores the importance of prompt design (Branon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024; He et al. 2024). Other work uses LLMs to enhance node features and graph structures, to act as predictors, or to serve as feature extractors for GNNs (Chen et al. 2023a; Chandra et al. 2020; Chen et al. 2023b; Mavromatis et al. 2023); for inference, Zhu et al. (2024) combined LLMs and GNNs via fine-tuning on textual graphs, and Duan et al. (2023) introduced an effective approach for textual graph learning. While straightforward, these methods struggle to fully capture structural intricacies (especially for large graphs) and depend heavily on prompt engineering.

Graph Representation as a Learned Projector. The second family addresses those limitations by transforming graph data into embeddings or graph vectors that LLMs process more directly. It includes methods like LLAGA, which uses node-level templates and a linear projector (Chen et al. 2024), and others that inject GNN layers into a pre-trained LLM (Liu et al. 2024; Qin et al. 2023). Related approaches use graph neural prompts to encode knowledge graphs (Tian et al. 2024), build prompt-based node feature extractors with a GNN adapter (Huang et al. 2023b), or condense graph information into fixed-length prefixes using graph transformers (Chai et al. 2023; Yang et al. 2021; Peng et al. 2024).

This Work: Guided by the taxonomy above, we scope our robustness study to projector methods built on *frozen pre-trained LLMs*, because they are computationally efficient and avoid the substantial costs of retraining (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). As representatives of the two projector designs, we study LLAGA (Chen et al. 2024), which employs node-level templates with a linear projector, and GRAPH-PROMPTER (Liu et al. 2024), which uses a GNN with a linear projector. We set aside textual description methods, since they struggle with scalability and do not introduce the graph-specific vulnerabilities that are the focus of our contributions.

3 Threat Model

To make our robustness study concrete and comparable to prior work, we adopt a threat model grounded in established research on adversarial attacks in graphs and extend it to graph-aware LLMs (Zügner, Akbarnejad, and Günnemann 2018; Wu et al. 2019; Zügner and Günnemann 2019; Ma, Ding, and Mei 2020; Jin et al. 2021; Dai et al. 2023; Li et al. 2023). The model reflects realistic, low-access adversaries. In social networks, for instance, attackers often control only a few bot accounts; this limited access matches our setting, where an adversary manipulates a small subset of nodes to evade detection or influence legitimate user classification.

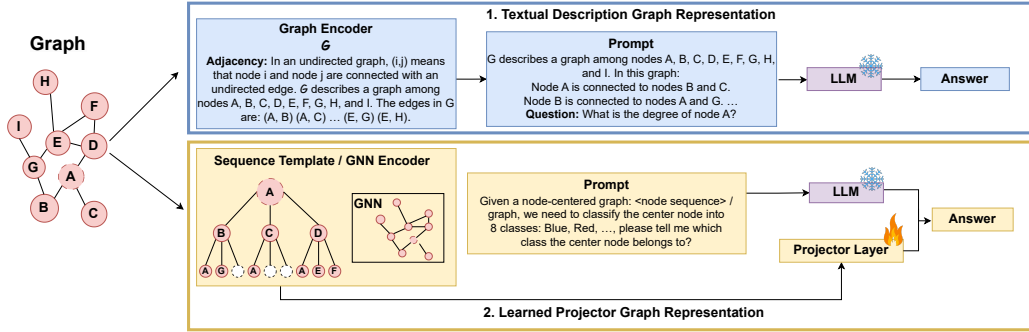


Figure 1: **Graph encoding-based adaptations of LLMs.** Our taxonomy separates two encoding families. In the textual description representation (top), graphs are explained in natural language. In the learned projector representation (bottom), graphs are encoded either by node templates that turn the graph into sequences or by GNNs. In both families the LLM is frozen and only the projector that maps the graph into tokens is trained – a distinction that, as our study shows, directly shapes where each model is vulnerable.

Similarly, on Wikipedia, hoax articles with sparse connections to legitimate ones can be strategically linked to induce misclassification of genuine content. Such transferability is especially concerning because graph-aware LLMs are candidates for deployment in critical applications such as recommendation systems, social media analysis, and healthcare, potentially replacing GNNs. Following prior work, we characterize the threat model along three dimensions – model access, adversarial capabilities, and attack strategies – and then state the attacker’s goal.

Model Access. The attacker operates in a black-box setting with no knowledge of the model’s architecture or parameters, and can influence the model only indirectly by interacting with the training or test data.

Adversarial Capabilities. Consistent with existing poisoning (evasion) attacks, the attacker has full access to the training (inference) data of the target model, and can modify or inject data within a predefined budget under unnoticeability constraints that aim to avoid detection.

Attack Strategies. The attacker uses two primary strategies: feature manipulation (altering node features) and structure manipulation (adding or removing edges) – the two axes along which our contributions evaluate vulnerability.

Attacker’s Goal. The attacker aims to compromise model integrity by degrading overall performance, specifically by increasing the misclassification rate of the classification model.

4 Experimental Setup

Dataset. To evaluate our contributions on standard benchmarks, we use three widely used text-attributed graph datasets that vary in size and sparsity from small to medium scale: Cora, Citeseer, and PubMed (Yang, Cohen, and Salakhudinov 2016), all common benchmarks for models like those in our study (Chen et al. 2023a, 2024; Liu et al. 2024). To further test scalability, we additionally include the larger ArXiv dataset (Hu et al. 2020) (see Appendix A.5). We adopt standard train/validation/test splits of 6:2:2 for the three smaller datasets and 5:2:3 for ArXiv, following (Chen et al. 2024). Detailed statistics are provided in Table 1.

Table 1: Statistics of the benchmark datasets used throughout our study.

Data	#Node	#Edge	Sparsity (%)
Cora	2708	5429	14.81
CiteSeer	3327	4614	8.34
PubMed	19717	44338	2.28

Tasks. Our primary task is *node classification*, a widely used setting for evaluating graph machine learning models and the standard target for adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Liu and Wu 2023).

Evaluation Metrics. We report *accuracy*, the proportion of correctly predicted labels. To ensure the reliability of the comparisons underlying our contributions, each experiment is repeated three times and we report the mean and standard deviation. All experiments were conducted on a machine equipped with an NVIDIA A40 GPU with 48GB of memory.

Choice of Graph-aware LLM and Adversarial Attack Methods. We now justify the graph-aware LLM methods and adversarial attack methods that underpin our robustness study, chosen to make each model’s design choices directly comparable.

Graph-Aware LLM Methods. As the two representatives of the projector family identified by our taxonomy, we study LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024). Both build on a *frozen pre-trained LLM*, which makes them computationally efficient and adaptable across graph tasks and LLM architectures, and both differ mainly in how they encode structure: LLAGA uses node-level templates with a linear projector, while GRAPHPROMPTER employs a GNN-based encoder with a linear projector. In both cases the projector is a 2-layer multi-layer perceptron (MLP). The foundational LLMs for LLAGA and GRAPHPROMPTER are Vicuna-7B (Chiang et al. 2023) and LLAMA2-7B (Touvron et al. 2023), respectively.

Table 2: Contribution 1 – transferring GNN attacks to graph-aware LLMs. Accuracy of LLAGA and GRAPHPROMPTER under NETTACK and METAATTACK poisoning and evasion attacks on Cora, Citeseer, and PubMed (percentage drop from clean accuracy in parentheses). Evasion attacks are far more damaging than poisoning, and LLAGA is consistently the more vulnerable model.

Attack	Type	Cora		Citeseer		PubMed	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Clean	—	0.89 $\pm$ 0.07	0.60 $\pm$ 0.03	0.71 $\pm$ 0.04	0.70 $\pm$ 0.03	0.90 $\pm$ 0.04	0.90 $\pm$ 0.02
NETTACK	Poisoning	0.87 $\pm$ 0.04 (2%)	0.60 $\pm$ 0.03 (0%)	0.64 $\pm$ 0.05 (10%)	0.68 $\pm$ 0.02 (3%)	0.89 $\pm$ 0.03 (1%)	0.88 $\pm$ 0.02 (2%)
	Evasion	0.55 $\pm$ 0.09 (38%)	0.53 $\pm$ 0.05 (12%)	0.59 $\pm$ 0.04 (19%)	0.61 $\pm$ 0.03 (14%)	0.84 $\pm$ 0.05 (7%)	0.81 $\pm$ 0.02 (10%)
METAATTACK	Poisoning	0.79 $\pm$ 0.03 (11%)	0.58 $\pm$ 0.05 (3%)	0.63 $\pm$ 0.04 (12%)	0.65 $\pm$ 0.06 (7%)	0.89 $\pm$ 0.08 (1%)	0.85 $\pm$ 0.02 (6%)
	Evasion	0.44 $\pm$ 0.06 (51%)	0.52 $\pm$ 0.04 (13%)	0.50 $\pm$ 0.03 (35%)	0.56 $\pm$ 0.02 (22%)	0.73 $\pm$ 0.04 (19%)	0.77 $\pm$ 0.03 (14%)

Adversarial Attack Methods. For the transfer study (Contribution 1), we adopt two representative structural attacks, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), in both poisoning and evasion modes. NETTACK is a targeted attack that finds minimal perturbations to misclassify specific nodes, while METAATTACK performs untargeted attacks by treating the graph structure as a hyperparameter to optimize. Following (Li et al. 2023; Zügner and Günnemann 2019), we convert NETTACK into an untargeted evaluation by sequentially targeting each node and aggregating the results. For all experiments we perturb only 10% of the edges to satisfy the unnoticeability constraint, as in (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019). Full details are in Appendix A.2.

5 Results

5.1 Poisoning and Evasion Attacks on Graph-aware LLMs

This section delivers our first contribution: quantifying how well established GNN attacks transfer to graph-aware LLMs. We adapt NETTACK (Zügner and Günnemann 2019) and METAATTACK (Zügner and Günnemann 2019) to both models. For poisoning attacks on LLAGA, we use the perturbed graph to generate the node-level sequences and then retrain the projector; for GRAPHPROMPTER, we re-encode the perturbed graph with a GAT and retrain its projector. For evasion attacks, both models are trained on unperturbed data, and perturbations are introduced only at inference.

Results. Table 2 yields three takeaways. **(i) Poisoning barely dents these models, but evasion breaks them.** Poisoning causes at most an 11% drop across both attacks, suggesting that the models’ robust representations and the LLM’s additional context provide resilience; evasion, by contrast, is highly effective, degrading accuracy by up to 51%. **(ii) METAATTACK is the stronger attack.** It generally outperforms NETTACK for both poisoning and evasion across all datasets, thanks to its ability to craft small but highly effective perturbations by treating graph edges as hyperparameters. **(iii) Denser graphs are more vulnerable.** For example, LLAGA loses 51% accuracy under evasion on the denser Cora but only 19% on the sparser PubMed. These observations set up the deeper question we address next: *why* is one

paradigm more exposed than the other.

5.2 Which Graph-aware Paradigm is More Susceptible to Graph Adversarial Attacks and Why?

A key insight of our study – and direct support for the encoding-based taxonomy (Contribution 2) – is that *how a model encodes structure predicts how vulnerable it is*. Table 2 shows that LLAGA is consistently more susceptible than GRAPHPROMPTER, and the reason lies in its *sensitivity to structural perturbations*. LLAGA integrates the graph structure in two ways: by generating a fixed-length node-level sequence for each node and by computing a graph Laplacian embedding. Each position in this sequence corresponds to a specific relative structural position in the original graph, so because the node embeddings are derived directly from the perturbed structure, the attack propagates straight into the information the LLM processes, driving performance down. GRAPHPROMPTER is more robust precisely because it encodes structure differently: its textual node embeddings are untouched by structural perturbations, since only the GNN component is affected, and the intact textual features augment the perturbed structure to preserve accuracy.

6 Node Sequence Template Injection Attacks

Our third contribution is the discovery of an attack surface unique to LLAGA, exposed by the neighborhood detail template it uses to construct node sequences. The attack is illustrated in Figure 2, with the algorithm in Appendix A.4. To obtain the node sequence for a node u , the neighborhood detail template in LLAGA first constructs a *fixed-shape* computational tree, where the children of a node are its sampled 1-hop neighbors, repeated for each subsequent level. When a node has too few neighbors, a placeholder fills the gap to maintain the fixed size of, e.g., $k = 10$ children, consistent with LLAGA’s original design (see “Clean” in Figure 2 and lines 1-14 of the algorithm). We show that this seemingly innocuous fixed structure is exactly what opens a new attack surface.

Concretely, by manipulating the graph, an attacker can inject malicious nodes into these placeholder slots and significantly degrade model performance. We instantiate the attack with three strategies of increasing strength. *Non-Adjacent*

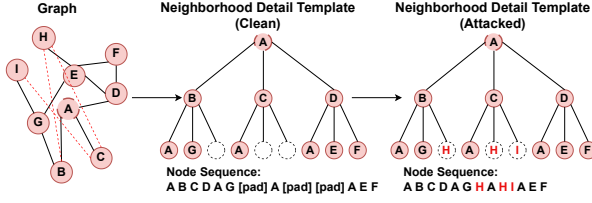


Figure 2: Contribution 3 – the node sequence template injection attack, a new attack surface we uncover in LLAGA. The example shows the Non-adjacent Injection attack (NI); red dashed lines are the attacker’s edge perturbations. For Supernode Injection (SI) and Multiple Supernode Injection (MSI), the supernode(s) take the place of the placeholders. Here $k = 3$.

Injection (NI) injects nodes that lie outside the 2-hop neighborhood of a target node u (see ”Attacked” in Figure 2 and lines 21-27). *Supernode Injection (SI)* injects a high-degree node as a placeholder, again outside the target’s 2-hop neighborhood. *Multiple Supernode Injection (MSI)* extends *SI* by injecting multiple distinct supernodes as placeholders (lines 35-43). In all three, inserting uninformative nodes into the node sequence corrupts the target node’s representation, yielding a poorly trained projection layer and compromising both learning and inference.

Results. Table 3 confirms that this new attack surface is dangerous. LLAGA stays resilient to poisoning on Cora and Citeseer but collapses under evasion, with accuracy falling to 0.42, 0.36, and 0.30 on Cora and to 0.36, 0.30, and 0.24 on Citeseer for the *NI*, *SI*, and *MSI* attacks, respectively – the ordering confirming that stronger injections do more damage.

The larger PubMed graph is more resilient, with only a modest evasion drop (0.85, 0.82, and 0.76). The reason is scale: in smaller, tightly connected graphs like Cora and Citeseer a localized injection has a concentrated impact that severely disrupts local-neighborhood reasoning, whereas on the much larger PubMed graph the same attack is diluted.

Crucially, this new attack surface is more damaging than the transferred structural attacks of Contribution 1: *MSI* drives evasion accuracy down to 0.30, 0.24, and 0.76 on Cora, CiteSeer, and PubMed, versus 0.44, 0.50, and 0.73 for METAATTACK (Table 2) – underscoring that vulnerabilities specific to the LLM’s graph encoding can eclipse classical graph attacks.

7 Poisoning and Evasion Attack via Feature Perturbation

This section develops our fourth contribution: exposing the feature-perturbation vulnerability that the GNN literature has largely overlooked. Existing graph adversarial attacks focus on structural perturbations (Li et al. 2023; Wu et al. 2019; Dai et al. 2018; Sun et al. 2022; Zügner and Günnemann 2019), with the exception of NETTACK (Zügner, Akbarnejad, and Günnemann 2018), which does not transfer to the textual domain. Since structural attacks are only moderately successful on robust models like GRAPHPROMPTER, and

Table 3: Contribution 3 – accuracy of LLAGA under our node sequence template injection attack across datasets. *NI*, *SI*, and *MSI* denote the non-adjacent, single-supernode, and multiple-supernode injection attacks, respectively; Clean is the unattacked model. Evasion is markedly more effective than poisoning, and impact grows with attack strength ($NI < SI < MSI$).

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 ± 0.07	—
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	—	0.71 ± 0.04	—
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	—	0.90 ± 0.04	—
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03

since textual features are central to these models’ predictions, we ask whether imperceptible perturbations to the text itself can degrade performance.

We adapt the imperceptible attack strategies of (Boucher et al. 2022) with one key modification: we generate perturbations against a task-agnostic translation objective (English to French). The goal is to modify the input text so that the changes are visually identical yet produce an incorrect translation, corrupting the encoded semantics on the intuition that if the translation is wrong, the encoded semantics are wrong too. We use two imperceptible attacks: *Homoglyphs*, which replace characters with visually similar ones, and *Reorderings*, which use direction control characters to alter the rendering sequence. Examples appear in Appendix A.6.

Following (Boucher et al. 2022), we optimize the perturbations with differential evolution (Storn and Price 1997), maximizing the Levenshtein distance between the model’s output on the perturbed and unperturbed text, with a perturbation budget of 10% of the average length of each textual feature.

Results. Table 4 establishes the feature vulnerability. Reordering attacks are consistently more potent than Homoglyph attacks across all models and datasets. On Cora and CiteSeer, LLAGA is vulnerable to both, with poisoning drops of 27% and 23% under Homoglyph and a larger 35% and 25% under Reordering; GRAPHPROMPTER is again more resilient. PubMed follows the same trends, with a smaller overall impact than the smaller datasets.

The headline result of this contribution is a fundamental shift in the threat landscape, shown in Figure 3: feature perturbations (Reordering) outperform structural attacks (METAATTACK) by a wide margin on *both* LLAGA and GRAPHPROMPTER. In other words, graph-aware LLMs weigh input features more heavily than graph structure, making feature perturbation the dominant vulnerability. The next section pushes this insight further by combining both attack types.

Table 4: Contribution 4 – feature perturbation attacks (percentage drop from clean accuracy in parentheses). Reordering dominates Homoglyph, and both expose a feature-level vulnerability overlooked in the GNN literature.

Data	Method	Poisoning		Evasion	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Cora	Homoglyph	0.65 $\pm$ 0.05 (27%)	0.52 $\pm$ 0.04 (13%)	0.41 $\pm$ 0.04 (54%)	0.38 $\pm$ 0.03 (37%)
	Reordering	0.58 $\pm$ 0.06 (35%)	0.45 $\pm$ 0.05 (25%)	0.24 $\pm$ 0.05 (73%)	0.20 $\pm$ 0.03 (67%)
Citeseer	Homoglyph	0.54 $\pm$ 0.04 (23%)	0.65 $\pm$ 0.04 (7%)	0.32 $\pm$ 0.03 (55%)	0.52 $\pm$ 0.05 (26%)
	Reordering	0.46 $\pm$ 0.03 (35%)	0.39 $\pm$ 0.02 (44%)	0.20 $\pm$ 0.05 (72%)	0.27 $\pm$ 0.02 (61%)
PubMed	Homoglyph	0.72 $\pm$ 0.04 (20%)	0.83 $\pm$ 0.04 (8%)	0.36 $\pm$ 0.06 (60%)	0.55 $\pm$ 0.03 (39%)
	Reordering	0.67 $\pm$ 0.05 (26%)	0.80 $\pm$ 0.03 (11%)	0.28 $\pm$ 0.04 (69%)	0.42 $\pm$ 0.04 (53%)

Table 5: Model accuracy under our unified imperceptible attack that jointly perturbs structure and features (percentage drop from clean accuracy in parentheses). The combination is far more damaging than either attack alone.

Data	Attack	LLAGA	GRAPHPROMPTER
Cora	Poisoning	0.35 $\pm$ 0.05 (61%)	0.28 $\pm$ 0.08 (53%)
	Evasion	0.14 $\pm$ 0.06 (84%)	0.11 $\pm$ 0.05 (82%)
Citeseer	Poisoning	0.26 $\pm$ 0.04 (63%)	0.22 $\pm$ 0.03 (69%)
	Evasion	0.10 $\pm$ 0.09 (86%)	0.13 $\pm$ 0.07 (81%)
PubMed	Poisoning	0.39 $\pm$ 0.07 (57%)	0.32 $\pm$ 0.04 (64%)
	Evasion	0.12 $\pm$ 0.05 (87%)	0.19 $\pm$ 0.04 (79%)

8 A Unified Imperceptible Attack on Graph-aware LLMs

Building directly on the feature-vulnerability finding, we now show that combining feature and structural perturbations yields a strictly stronger, still-imperceptible attack. We hypothesize that jointly perturbing both channels degrades accuracy more than either alone while keeping alterations to structure and features imperceptible.

For the structural component we use METAATTACK, which subtly adjusts edges; for the feature component we combine the Homoglyph and Reordering attacks from the previous section. Together they form a highly imperceptible attack that simultaneously targets the relational and textual information on which graph-aware LLMs rely.

Results. Table 5 confirms the hypothesis: the unified attack is substantially more effective than either perturbation alone across all datasets, driving accuracy down by up to 87%. This delivers the sharpest evidence for our central architectural insight – unlike traditional GNNs, which are primarily sensitive to structural perturbations, graph-aware LLMs depend on the consistency of *both* structure and node features, and are therefore highly susceptible to combined, inconspicuous modifications. This same dual vulnerability is precisely what motivates the two-pronged defense we present next.

9 Ablation Study

9.1 Varying Feature Perturbation

To probe the robustness of our feature-vulnerability finding, we vary the fraction of perturbed features from 1% to 10%. Using the Reordering attack (which mirrors the Homoglyph pattern), we observe a consistent trend across all models and datasets (Cora, Citeseer, and PubMed): fewer perturbed features yield a weaker attack, as expected. Notably, as the perturbation level drops, the success rates of poisoning and evasion converge, because with fewer perturbations the adversarial signal becomes too weak to matter regardless of whether it was introduced at training (poisoning) or inference (evasion). A detailed plot is in the Appendix (Figure 2). This confirms that our reported vulnerabilities scale smoothly with the attack budget rather than depending on a single operating point.

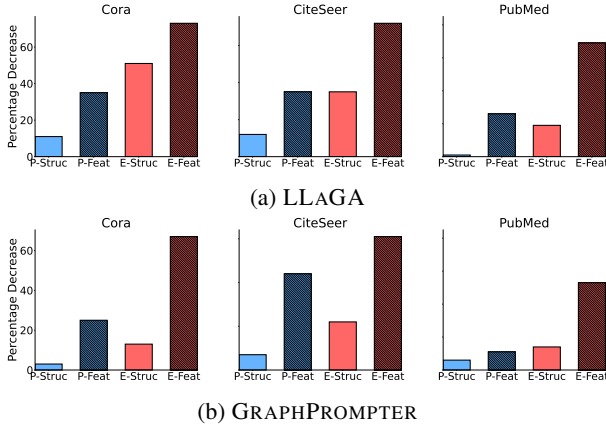


Figure 3: Structure- vs. feature-based adversarial attacks on graph-aware LLMs, the central comparison behind Contribution 4. Attack success is the percentage decrease (y-axis) from the original accuracy. P-Struc, P-Feat, E-Struc, E-Feat denote poisoning/evasion attacks under structural and feature perturbation. Feature attacks dominate structural ones for both models.

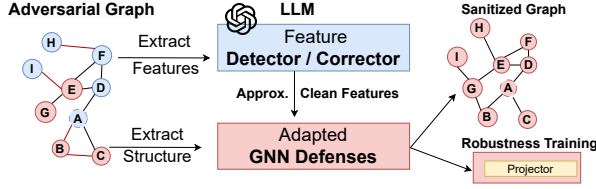


Figure 4: **Overview of our GALGUARD defense (Contribution 5).** Perturbed node features and edges are colored blue and red, respectively. The two prongs mirror the two vulnerabilities we identified: an LLM first detects and corrects perturbations in the node features extracted from an adversarial graph, and the refined features together with the perturbed edges are then passed through existing GNN defenses, which can either produce a sanitized graph or be adapted for robustness training of the projector.

9.2 Does the Choice of LLM Affect Attack Performance?

Finally, we check that our vulnerability findings are a property of the graph-encoding design rather than of a particular backbone LLM. LLAGA uses Vicuna-7B (Chiang et al. 2023) and GRAPH PROMPTER uses Llama2-7B (Touvron et al. 2023); we swap LLAGA’s backbone to Llama2-7B and compare under poisoning and evasion attacks on all datasets. Both backbones achieve similar clean accuracy (e.g., Vicuna: 0.89 ± 0.07 vs. Llama2: 0.90 ± 0.03 on Cora) and comparable vulnerability under attack, and the same holds when swapping the LLM in GRAPH PROMPTER. The choice of foundation LLM thus has minimal impact on adversarial robustness – reinforcing that the vulnerabilities we identify stem from how graphs are encoded, not from the underlying LLM. Full results are in Table 2 of the Appendix.

10 Defense: GALGUARD

Our final contribution turns the preceding attack analysis into a defense. The dual vulnerability we uncovered – to both structural and feature perturbations – is exactly what makes existing GNN defenses insufficient for graph-aware LLMs. Many such defenses prune or reweight edges based on node feature similarity (Wu et al. 2019; Dai et al. 2018; Zhang and Zitnik 2020; Jin et al. 2020; Zhang and Ma 2024), but they assume clean features and fail once features are perturbed (see Figure 3 in the Appendix). Likewise, defenses that decouple structure and features (Shanthamallu, Thiagarajan, and Spanias 2021; Wu et al. 2021), and even OCR-based sanitization (Boucher et al. 2022), break down under the coordinated attacks we introduced. This gap motivates a new, integrated defense.

We therefore propose graph-aware LLM defense (GALGUARD), an end-to-end strategy whose two prongs directly counter the two vulnerabilities: (1) an LLM-based feature corrector and (2) adapted GNN-based structural defenses.

Feature Robustness Strategy. This first prong targets the feature vulnerability of Contribution 4. As shown in Figure 4, an LLM acts as an adaptive mechanism that analyzes

textual node features for anomalous patterns and provides approximate corrections to restore feature integrity, mitigating feature-level perturbations – a practical remedy for feature vulnerabilities. The specific instructions and prompts are in Appendix A.6; we use GPT-4 Turbo as the LLM corrector.

Graph Structure Robustness Strategy. This second prong targets the structural vulnerability via a two-step approach. First, as a preprocessing step, we adopt a *graph purification heuristic* (Wu et al. 2019): for each node v we compute the cosine similarity between its embedding $\phi(x_v)$ and its neighbors’ embeddings $\phi(x_u)$ and remove low-similarity edges. Second, we integrate *GNNGuard* (Zhang and Zitnik 2020), applying its principles directly to the message-passing layers of GRAPH PROMPTER. For LLAGA, we contribute a novel adaptation: we fold the purification step into the construction of the computational tree for its node sequence, and introduce a learnable *global structural context embedding* (M_{global}) as a parameter in LLAGA’s projector. Concatenating M_{global} with the purified node sequence embedding and training them jointly teaches the model to operate robustly on perturbed graph inputs. GALGUARD is plug-and-play: other defenses, e.g., Pro-GNN (Jin et al. 2020), can replace GNNGuard.

10.1 Results

Table 6 reports accuracy under METAATTACK against two variants of our defense: GALGUARD_p, which combines the feature corrector with graph purification, and the full GALGUARD, which adds GNNGuard (or its LLAGA adaptation). The results validate the defense on both fronts. Across all datasets (Cora, CiteSeer, and PubMed) and both poisoning and evasion attacks, our defenses consistently mitigate METAATTACK. GALGUARD_p is already an effective foundation, showing that correcting features with the LLM and applying initial purification substantially improve robustness; the full GALGUARD consistently improves further, evidencing the synergy of adding the GNN-inspired structural defense. Together, this closes the loop on the vulnerabilities established earlier in the paper.

Table 6: Contribution 5 – accuracy under METAATTACK with and without our proposed defense GALGUARD, for poisoning and evasion attacks. GALGUARD_p combines feature correction with graph purification; the full GALGUARD adds the adapted GNN structural defense and yields the best robustness (bold) in every setting.

Data	Method	LLAGA		GRAPH PROMPTER	
		Poisoning	Evasion	Poisoning	Evasion
Cora	METAATTACK	0.79 $\pm$ 0.03	0.44 $\pm$ 0.06	0.58 $\pm$ 0.05	0.52 $\pm$ 0.04
	GALGUARD _p	0.82 $\pm$ 0.05	0.62 $\pm$ 0.04	0.59 $\pm$ 0.03	0.56 $\pm$ 0.04
	GALGUARD	0.85$\pm$0.04	0.83$\pm$0.04	0.60$\pm$0.05	0.59$\pm$0.03
CiteSeer	MetaAttack	0.63 $\pm$ 0.04	0.50 $\pm$ 0.03	0.65 $\pm$ 0.06	0.56 $\pm$ 0.03
	GALGUARD _p	0.64 $\pm$ 0.03	0.56 $\pm$ 0.02	0.67 $\pm$ 0.05	0.60 $\pm$ 0.06
	GALGUARD	0.67$\pm$0.04	0.62$\pm$0.04	0.68$\pm$0.04	0.64$\pm$0.05
PubMed	MetaAttack	0.89 $\pm$ 0.08	0.73 $\pm$ 0.04	0.85 $\pm$ 0.02	0.77 $\pm$ 0.03
	GALGUARD _p	0.90 $\pm$ 0.06	0.78 $\pm$ 0.03	0.85 $\pm$ 0.04	0.81 $\pm$ 0.04
	GALGUARD	0.90$\pm$0.01	0.87$\pm$0.02	0.89$\pm$0.02	0.89$\pm$0.01

11 Conclusion

This paper takes a substantial first step in uncovering the adversarial vulnerabilities of graph-aware LLMs, a largely unexplored area, and delivers five contributions. We provided the first systematic transfer of GNN attacks to graph-aware LLMs (Contribution 1) and an encoding-based taxonomy that explains why the paradigm shapes vulnerability (Contribution 2), showing that the methods used to integrate graph data are themselves a critical source of risk. We discovered a new attack surface – the node sequence template injection attack on LLaGA– demonstrating that design choices can sharply increase susceptibility (Contribution 3). We then showed that feature perturbation attacks are highly effective and, unlike in traditional GNNs, often outperform structural attacks, with a unified attack combining both able to render graph-aware LLMs near-useless (Contribution 4).

Finally, to counter these threats, we proposed GALGUARD, an end-to-end defense that pairs an LLM-based feature corrector with adapted GNN-based structural defenses (Contribution 5) and consistently restores robustness. As graph-aware LLMs spread into applications across social networks, healthcare, and finance, ensuring their resilience to adversarial exploitation is essential. We hope this work serves as a foundation for deeper study of the vulnerabilities and defenses of graph-aware LLMs, fostering both theoretical insight and practical safeguards for their real-world deployment.

References

- Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Ahmedt-Aristizabal, D.; Armin, M. A.; Denman, S.; Fookes, C.; and Petersson, L. 2021. Graph-based deep learning for medical diagnosis and analysis: past, present and future. *Sensors*, 21(14): 4758.
- Anil, R.; Dai, A. M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.
- Bi, B.; Liu, S.; Wang, Y.; Mei, L.; and Chen, X. 2024. Scalable Link Prediction on Large-Scale Heterogeneous Graphs with Large Language Models. *arXiv preprint arXiv:2401.13227*.
- Boucher, N.; Shumailov, I.; Anderson, R.; and Papernot, N. 2022. Bad characters: Imperceptible nlp attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, 1987–2004. IEEE.
- Brannon, W.; Fulay, S.; Jiang, H.; Kang, W.; Roy, B.; Kabbara, J.; and Roy, D. 2023. Congrat: Self-supervised contrastive pretraining for joint graph and text embeddings. *arXiv preprint arXiv:2305.14321*.
- Chai, Z.; Zhang, T.; Wu, L.; Han, K.; Hu, X.; Huang, X.; and Yang, Y. 2023. GraphLLM: Boosting Graph Reasoning Ability of Large Language Model. *arXiv preprint arXiv:2310.05845*.
- Chandra, S.; Mishra, P.; Yannakoudakis, H.; Nimishakavi, M.; Saeidi, M.; and Shutova, E. 2020. Graph-based Modeling of Online Communities for Fake News Detection. *arXiv preprint arXiv:2008.06274*.
- Chen, R.; Zhao, T.; Jaiswal, A.; Shah, N.; and Wang, Z. 2024. LLaGA: Large Language and Graph Assistant. In *Forty-first International Conference on Machine Learning*.
- Chen, Z.; Mao, H.; Li, H.; Jin, W.; Wen, H.; Wei, X.; Wang, S.; Yin, D.; Fan, W.; Liu, H.; and Tang, J. 2023a. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *arXiv preprint arXiv:2307.03393*.
- Chen, Z.; Mao, H.; Wen, H.; Han, H.; Jin, W.; Zhang, H.; Liu, H.; and Tang, J. 2023b. Label-free Node Classification on Graphs with Large Language Models (LLMs). *arXiv preprint arXiv:2310.04668*.
- Chiang, W.-L.; Li, Z.; Lin, Z.; Sheng, Y.; Wu, Z.; Zhang, H.; Zheng, L.; Zhuang, S.; Zhuang, Y.; Gonzalez, J. E.; Stoica, I.; and Xing, E. P. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.
- Conti, M.; Li, J.; Picek, S.; and Xu, J. 2022. Label-only membership inference attack against node-level graph neural networks. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, 1–12.
- Dai, E.; Lin, M.; Zhang, X.; and Wang, S. 2023. Unnoticeable backdoor attacks on graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 2263–2273.
- Dai, H.; Li, H.; Tian, T.; Huang, X.; Wang, L.; Zhu, J.; and Song, L. 2018. Adversarial attack on graph structured data. In *International conference on machine learning*, 1115–1124. PMLR.
- Dong, N.; Mücke, S.; and Khosla, M. 2022. Mucomid: A multitask graph convolutional learning framework for miRNA-disease association prediction. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 19(6): 3081–3092.
- Duan, K.; Liu, Q.; Chua, T.-S.; Yan, S.; Ooi, W. T.; Xie, Q.; and He, J. 2023. SimTeG: A Frustratingly Simple Approach Improves Textual Graph Learning. *arXiv preprint arXiv:2308.02565*.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Duddu, V.; Boutet, A.; and Shejwalkar, V. 2020. Quantifying privacy leakage in graph embedding. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*, 76–85.
- Fan, W.; Ma, Y.; Li, Q.; He, Y.; Zhao, E.; Tang, J.; and Yin, D. 2019. Graph neural networks for social recommendation. In *The world wide web conference*, 417–426.
- Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. *arXiv preprint arXiv:2310.04560*.
- Gong, N. Z.; and Liu, B. 2018. Attribute inference attacks in online social networks. *ACM Transactions on Privacy and Security (TOPS)*, 21(1): 1–30.

- Guo, J.; Du, L.; Liu, H.; Zhou, M.; He, X.; and Han, S. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data ? An Empirical Evaluation and Benchmarking. *arXiv preprint arXiv:2305.15066*.
- Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30.
- He, X.; Bresson, X.; Laurent, T.; Perold, A.; LeCun, Y.; and Hooi, B. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*.
- He, X.; Wen, R.; Wu, Y.; Backes, M.; Shen, Y.; and Zhang, Y. 2021. Node-level membership inference attacks against graph neural networks. *arXiv preprint arXiv:2102.05429*.
- Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; and Leskovec, J. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33: 22118–22133.
- Huang, J.; Zhang, X.; Mei, Q.; and Ma, J. 2023a. Can LLMs Effectively Leverage Structural Information for Graph Learning: When and Why. *arXiv preprint arXiv:2309.16595*.
- Huang, X.; Han, K.; Bao, D.; Tao, Q.; Zhang, Z.; Yang, Y.; and Zhu, Q. 2023b. Prompt-based Node Feature Extractor for Few-shot Learning on Text-Attributed Graphs. *arXiv preprint arXiv:2309.02848*.
- Jin, W.; Li, Y.; Xu, H.; Wang, Y.; Ji, S.; Aggarwal, C.; and Tang, J. 2021. Adversarial attacks and defenses on graphs. *ACM SIGKDD Explorations Newsletter*, 22(2): 19–34.
- Jin, W.; Ma, Y.; Liu, X.; Tang, X.; Wang, S.; and Tang, J. 2020. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 66–74.
- Kipf, T. N.; and Welling, M. 2022. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- Kuang, H.; Xu, J.; Zhang, H.; Zhao, Z.; Zhang, Q.; Huang, X.; and Wei, Z. 2023. Unleashing the Power of Language Models in Text-Attributed Graph. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 8429–8441. Association for Computational Linguistics.
- Li, K.; Liu, Y.; Ao, X.; and He, Q. 2023. Revisiting graph adversarial attack and defense from a data distribution perspective. In *The Eleventh International Conference on Learning Representations*.
- Liu, C.; and Wu, B. 2023. Evaluating Large Language Models on Graphs: Performance Insights and Comparative Analysis. *arXiv preprint arXiv:2308.11224*.
- Liu, S.; Nie, W.; Wang, C.; Lu, J.; Qiao, Z.; Liu, L.; Tang, J.; Xiao, C.; and Anandkumar, A. 2023. Multi-modal molecule structure–text model for text-based retrieval and editing. *Nature Machine Intelligence*, 5(12): 1447–1457.
- Liu, Z.; He, X.; Tian, Y.; and Chawla, N. V. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM on Web Conference 2024*, 481–484.
- Ma, J.; Ding, S.; and Mei, Q. 2020. Towards more practical adversarial attacks on graph neural networks. *Advances in neural information processing systems*, 33: 4756–4766.
- Mavromatis, C.; Ioannidis, V. N.; Wang, S.; Zheng, D.; Adeshina, S.; Ma, J.; Zhao, H.; Faloutsos, C.; and Karypis, G. 2023. Train your own GNN teacher: Graph-aware distillation on textual graphs. *arXiv preprint arXiv:2304.10668*.
- Olatunji, I. E.; Hizber, A.; Sihlovec, O.; and Khosla, M. 2023a. Does black-box attribute inference attacks on graph neural networks constitute privacy risk? *arXiv preprint arXiv:2306.00578*.
- Olatunji, I. E.; Nejd, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 11–20. IEEE.
- Olatunji, I. E.; Rathee, M.; Funke, T.; and Khosla, M. 2023b. Private Graph Extraction via Feature Explanations. *Proceedings on Privacy Enhancing Technologies*.
- Pan, S.; Luo, L.; Wang, Y.; Chen, C.; Wang, J.; and Wu, X. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*.
- Peng, Y.; Lin, S.; Chen, Q.; Xu, L.; Ren, X.; Li, Y.; and Xu, J. 2024. ChatGraph: Chat with Your Graphs. *arXiv preprint arXiv:2401.12672*.
- Qin, Y.; Wang, X.; Zhang, Z.; and Zhu, W. 2023. Disentangled Representation Learning with Large Language Models for Text-Attributed Graphs. *arXiv preprint arXiv:2310.18152*.
- Shanthamallu, U. S.; Thiagarajan, J. J.; and Spanias, A. 2021. Uncertainty-matching graph neural networks to defend against poisoning attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 9524–9532.
- Storn, R.; and Price, K. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11: 341–359.
- Su, B.; Du, D.; Yang, Z.; Zhou, Y.; Li, J.; Rao, A.; Sun, H.; Lu, Z.; and Wen, J.-R. 2022. A molecular multimodal foundation model associating molecule graphs with natural language. *arXiv preprint arXiv:2209.05481*.
- Sun, L.; Dou, Y.; Yang, C.; Zhang, K.; Wang, J.; Philip, S. Y.; He, L.; and Li, B. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(8): 7693–7711.
- Sun, S.; Ren, Y.; Ma, C.; and Zhang, X. 2023. Large Language Models as Topological Structure Enhancers for Text-Attributed Graphs. *arXiv preprint arXiv:2311.14324*.
- Tan, Y.; Zhou, Z.; Lv, H.; Liu, W.; and Yang, C. 2023. WalkLM: A Uniform Language Model Fine-tuning Framework for Attributed Graph Embedding. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.

- Tian, Y.; Song, H.; Wang, Z.; Wang, H.; Hu, Z.; Wang, F.; Chawla, N. V.; and Xu, P. 2024. Graph neural prompting with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 19080–19088.
- Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- Wang, J.; Zhang, S.; Xiao, Y.; and Song, R. 2021. A review on graph neural network methods in financial applications. *arXiv preprint arXiv:2111.15367*.
- Wang, X.; and Wang, W. H. 2022. Group property inference attacks against graph neural networks. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2871–2884.
- Wei, Y.; Fu, S.; Jiang, W.; Kwok, J. T.; and Zhang, Y. 2024a. Rendering graphs for graph reasoning in multimodal large language models. *arXiv preprint arXiv:2402.02130*.
- Wei, Y.; Huang, Q.; Kwok, J. T.; and Zhang, Y. 2024b. Kicgpt: Large language model with knowledge in context for knowledge graph completion. *arXiv preprint arXiv:2402.02389*.
- Wenkel, F.; Wolf, G.; and Knyazev, B. 2023. Pretrained Language Models to Solve Graph Tasks in Natural Language. In *ICML workshop on Structured Probabilistic Inference & Generative Modeling*.
- Wu, H.; Wang, C.; Tyshetskiy, Y.; Docherty, A.; Lu, K.; and Zhu, L. 2019. Adversarial examples on graph data: Deep insights into attack and defense. *arXiv preprint arXiv:1903.01610*.
- Wu, X.; Wu, H.; Zhou, X.; and Lu, K. 2021. CoG: a Two-View Co-training Framework for Defending Adversarial Attacks on Graph. *arXiv preprint arXiv:2109.05558*.
- Xie, H.; Zheng, D.; Ma, J.; Zhang, H.; Ioannidis, V. N.; Song, X.; Ping, Q.; Wang, S.; Yang, C.; Xu, Y.; et al. 2023. Graph-aware language model pre-training on a large graph corpus can help multiple graph applications. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5270–5281.
- Yang, J.; Liu, Z.; Xiao, S.; Li, C.; Lian, D.; Agrawal, S.; Singh, A.; Sun, G.; and Xie, X. 2021. GraphFormers: GNN-nested Transformers for Representation Learning on Textual graph. In *Proceedings of the 35th Conference on Neural Information Processing Systems*.
- Yang, Z.; Cohen, W.; and Salakhudinov, R. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, 40–48. PMLR.
- Ye, R.; Zhang, C.; Wang, R.; Xu, S.; and Zhang, Y. 2023. Natural Language is All a Graph Needs. *arXiv preprint arXiv:2308.07134*.
- Zhang, A.; and Ma, J. 2024. Defensevgae: Defending against adversarial attacks on graph data via a variational graph autoencoder. In *International Conference on Intelligent Computing*, 313–324. Springer.
- Zhang, J. 2023. Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt. *arXiv preprint arXiv:2304.11116*.
- Zhang, X.; and Zitnik, M. 2020. Gnn-guard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 33: 9263–9275.
- Zhang, Z.; Chen, M.; Backes, M.; Shen, Y.; and Zhang, Y. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium (USENIX Security 22)*, 4543–4560.
- Zhang, Z.; Liu, Q.; Huang, Z.; Wang, H.; Lu, C.; Liu, C.; and Chen, E. 2021. Graphmi: Extracting private graph data from graph neural networks. *arXiv preprint arXiv:2106.02820*.
- Zhang, Z.; Wang, X.; Zhang, Z.; Li, H.; Qin, Y.; and Zhu, W. 2023. LLM4DyG: Can Large Language Models Solve Problems on Dynamic Graphs? *arXiv preprint arXiv:2310.17110*.
- Zhao, J.; Zhuo, L.; Shen, Y.; Qu, M.; Liu, K.; Bronstein, M.; Zhu, Z.; and Tang, J. 2023. GraphText: Graph Reasoning in Text Space. *arXiv preprint arXiv:2310.01089*.
- Zhu, Y.; Wang, Y.; Shi, H.; and Tang, S. 2024. Efficient tuning and inference for large language models on textual graphs. *arXiv preprint arXiv:2401.15569*.
- Zügner, D.; Akbarnejad, A.; and Günnemann, S. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2847–2856.
- Zügner, D.; and Günnemann, S. 2019. Adversarial Attacks on Graph Neural Networks via Meta Learning. In *International Conference on Learning Representations (ICLR)*.

A Appendix

Organization: The Appendix expands on the material behind each contribution. Appendix A.1 gives the detailed encoding-based taxonomy (Contribution 2), and Appendix A.2 reviews related work on attacks against Graph Neural Networks (GNNs). Appendix A.3 details the NETTACK and METAATTACK methods used in our transfer study (Contribution 1), while Appendix A.4 presents the algorithm for the node sequence injection attack on LLAGA (Contribution 3). Appendix A.5 reports our ablation studies – varying the amount of perturbed features and the choice of base LLM. Finally, Appendix A.7 shows feature-attack examples and the LLM-corrector template supporting our defense (Contribution 5), and Appendix A.8 discusses the broader implications of our findings.

Code. Our code will be released upon acceptance.

A.1 Graph Encoding-Based Taxonomy of LLM Adaptations

Integrating large language models (LLMs) with graph data is an emerging area, with many approaches adapting or enhancing LLMs for graph tasks such as reasoning, representation, and inference. Our taxonomy (Contribution 2) organizes these approaches into two categories by how they encode graphs for LLMs: *textual descriptions of graphs*, where graph information is encoded as natural language, and *learned graph projector*, where a projector encodes graph data into embeddings or structured forms that LLMs can directly process. This section details the methods in each category; Figure 5 illustrates both approaches and Table 7 summarizes their differences. This organization is not merely descriptive: it is the lens through which we explain *why* the two representative models differ in robustness.

Graph Representation as a Textual Description In this first category, graph data is represented as natural language, allowing LLMs to process graph structures as textual descriptions. This approach has been explored extensively with works such as (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a) converting graph data into textual formats, often paired with specific instructions to query LLMs. For example, Tan et al. (2023) introduced a unified framework for attributed graph embedding by generating random walk-based textual sequences from graphs, which are then used to fine-tune LLMs. Similarly, Zhao et al. (2023) focused on graph reasoning within text spaces, translating graph structures into natural language to facilitate LLM interaction. Extending this idea, Wei et al. (2024a) augmented textual representations of graphs with visual representations of the graph, enhancing the reasoning capabilities of LLMs on graph data. In multimodal applications, Liu et al. (2023) and Su et al. (2022) linked molecule graphs with textual descriptions, enabling advances in drug discovery and molecular retrieval. Furthermore, Huang et al. (2023a) investigated how incorporating graph structure into LLM prompts can enhance predictive performance. Their findings revealed that LLMs tend to process graph-related prompts as contextual

paragraphs, rather than explicitly recognizing graph structures. This further underscores the importance of prompt design, as seen in studies like (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024), which extended LLMs to heterogeneous and large-scale graph tasks. Complementing these efforts, (He et al. 2024) explored using LLM-generated explanations as features to enhance both the performance and interpretability of LLMs on graphs.

While still representing graphs as textual description, another line of work integrates pretraining strategies with GNNs to enhance the LLMs’ graph reasoning capabilities. For instance, Chen et al. (2023a) proposed two LLM-GNN integration approaches: 1) using LLMs to enhance node features for node classification, and 2) employing LLMs as direct predictors for graph tasks. In line with this, Chandra et al. (2020) utilized LLMs as text encoders and GNNs as graph structure encoders for tasks such as fake news detection in online communities. Similarly, Chen et al. (2023b) explored the possibility of performing node classification without explicit labels by using LLMs to annotate nodes, which are then used in training a GNN model. Likewise, Mavromatis et al. (2023) utilized LLM as a feature extractor for GNNs.

For inference, Zhu et al. (2024) proposed a fine-tuning method for textual graph that combined LLMs and GNNs through a tunable side structure alongside each layer of the LLM, enhancing both training efficiency and inference speed. Similarly, Duan et al. (2023) introduced SimTeG, a simple yet effective approach that enhances textual graph learning by leveraging LLMs. In their method, the LLM is first fine-tuned on a downstream task, such as node classification. Afterward, node embeddings are generated by extracting the last hidden states from the fine-tuned LLM. Additionally, Zhang (2023) proposed Graph-toolformer, which leverages prompt engineering, powered by ChatGPT, to augment LLMs’ graph reasoning capabilities.

While these approaches make LLMs more accessible for graph tasks, they share two limitations that matter for our study: they often fail to fully capture the structural intricacies of graph data, and their performance depends heavily on prompt engineering, a process that requires careful, iterative design to work well.

Graph Representation as a Learned Projector To address the challenges posed by textual representation approaches, learned projector methods offer an alternative by transforming graph data into embeddings or graph vectors that LLMs can process more directly. For instance, Chen et al. (2024) introduced LLAGA, a large language and graph assistant that retains the general-purpose nature of LLMs while transforming graph structures into graph tokens, a format suitable for LLM processing. Similarly, (Liu et al. 2024; Qin et al. 2023) incorporated graph structure information into a pre-trained LLM through a tailored disentangled graph neural network layers. Tian et al. (2024) proposed using graph neural prompts to encode knowledge graph, transforming them into graph embeddings that LLMs can leverage during inference. Furthermore, Huang et al. (2023b) introduced a prompt-based node feature extractor (G-Prompt) for few-shot learning on text-attributed graphs. Their method combines

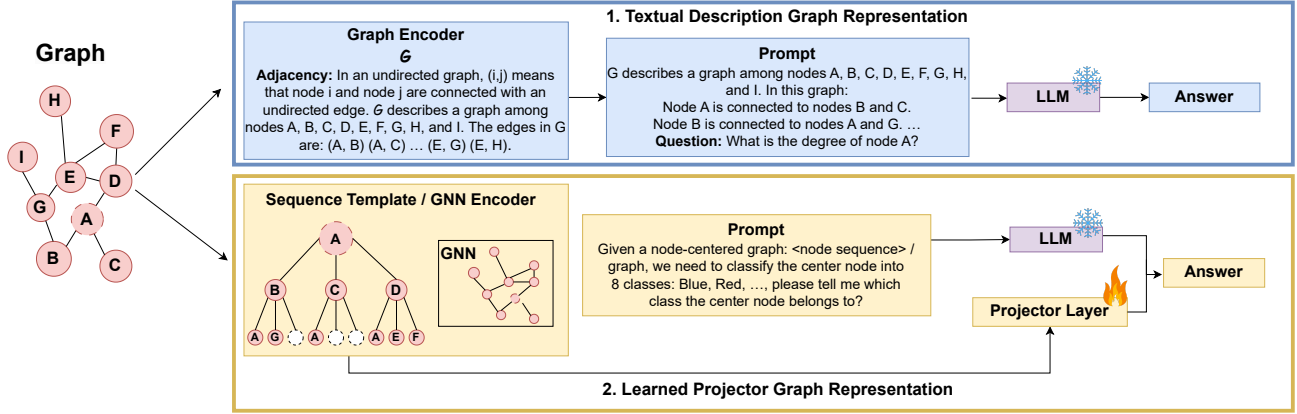


Figure 5: **Graph encoding-based adaptations of LLMs (our taxonomy, Contribution 2)**. In the textual description representation (top), graphs are explained in natural language. In the learned projector representation (bottom), graphs are encoded either by node templates that turn the graph into sequences or by GNNs. In both cases the LLM is frozen and only the projector that maps the graph into tokens is trained.

Table 7: Comparison of graph encoding-based adaptations of LLMs.

Aspect	Textual Description Approach	Learned Projector Approach
Graph Encoding	Converts graph data (nodes, edges) into natural language descriptions.	Uses a learned projector to encode graph structures into embeddings or graph tokens.
LLM Modifications	No modification to the LLM.	No modification to the LLM.
Graph Structure Representation	Describes graph topology and relationships using textual narratives.	Encodes relational and structural information directly into vector representations.
Flexibility	Can only be used for simple graphs.	More flexible in representing complex graph structures.
Computational Cost	No additional training is required. Lower cost.	Lightweight projector is trained, typically a linear layer. Minimal cost.
Suitability for Graph Tasks	May lose some relational nuance due to limitations in textual descriptions.	Captures detailed structural information, improving performance on graph-specific tasks.

a graph adapter (a learnable GNN layer) with task-specific prompts to capture neighborhood information and extract informative node features.

Another promising line of work condenses graph information into fixed-length prefixes using graph transformers, which are attached to each layer of the LLM to enhance its reasoning capabilities (Chai et al. 2023). Similarly, Yang et al. (2021) developed GraphFormers which seamlessly integrates text encoding and graph aggregation in an iterative workflow by nesting GNNs alongside the transformer blocks of a language model. Additionally, Peng et al. (2024) developed ChatGraph, an interactive interface that allows users to interact with their graphs through natural language. Here, graphs are represented as sequential paths and the LLM is fine-tuned on the graph data to support graph analysis.

We exclude projector adaptations that require retraining the LLM, since they sharply increase computational cost without substantial performance gains, and focus instead on adaptations built on **frozen pre-trained LLMs** (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). Beyond lower cost, this choice offers flexibility across LLM backbones and lets us exploit the LLMs’ extensive knowledge without retraining. As the two representatives that anchor our study, we use **LLAGA** (Chen et al. 2024), which pairs node-level templates with a linear projector, and **GRAPHPROMPTER** (Liu et al. 2024), which uses a GNN-based projector – a contrast that, as our results show, is exactly what separates the vulnerable from the more robust design.

This work. We deliberately scope our study to projector methods and set textual description methods aside, for two reasons. First, on capability: textual methods struggle to capture the structural intricacies of graph data, are largely limited to simple tasks such as describing basic properties like node degree, and become infeasible on larger graphs, where prompt token limits quickly bar the detailed descriptions required. Second, and decisive for our robustness question: textual methods introduce no unique attack surface – since the graph is rendered as plain text (e.g., “Node A is connected to Node B”), they are exposed to conventional text-based adversarial attacks rather than the structural or feature perturbations specific to graph models. They therefore lack the graph-specific vulnerabilities that motivate our investigation, which is why our contributions center on projector methods.

A.2 Attacks on GNN

Several types of attacks exist on GNNs, including membership inference attacks (Olatunji, Nejdli, and Khosla 2021; He et al. 2021; Conti et al. 2022), which aim to determine whether a specific node was part of the model’s training set; attribute inference attacks (Gong and Liu 2018; Olatunji et al. 2023a; Duddu, Boutet, and Shejwalkar 2020), which exploit the GNN’s output and learned embeddings to infer sensitive or missing node attributes; property inference attacks (Zhang et al. 2022; Wang and Wang 2022), which seek to extract global properties of the entire graph; adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Li et al. 2023; Dai et al. 2018; Sun et al. 2022), which attempt to degrade the model’s perfor-

mance by introducing subtle perturbations to the graph or node features; and graph reconstruction attacks (Zhang et al. 2021; Olatunji et al. 2023b), which aim to reconstruct the underlying graph structure using the learned embeddings or model outputs.

Among these attack types, adversarial attacks are of particular concern due to their direct impact on the model’s performance. In adversarial attacks, small, carefully crafted *perturbations* are introduced into the graph, such as modifying node features or altering edges, with the goal of significantly degrading the GNN’s performance. These attacks can be categorized into two types: **poisoning attacks**, which occur during the *training phase* by manipulating the training data, and **evasion attacks**, which occur at *test time*, where the adversary introduces perturbations to the input graph to mislead the model’s predictions. For instance, Zügner, Akbarnejad, and Günnemann (2018) demonstrated that adversarial perturbations on graph structures or node features can significantly degrade the performance of GNNs. Their method, **NETTACK**, employed a greedy optimization technique to compute the minimal perturbations that would maximize the model’s loss function during training. This approach effectively identifies the smallest changes in the features or graph structure needed to disrupt the model’s predictions. Li et al. (2023) revisited the problem of adversarial attacks on graph data from a data distribution perspective, highlighting how adversarial examples can significantly alter the underlying data distribution and proposed a novel defense mechanism to mitigate such effects. Similarly, Wu et al. (2019) provided a comprehensive analysis of adversarial examples on graph data, offering deep insights into both attacks and defenses. They focused on the vulnerabilities of GNNs to small perturbations and proposed defense techniques designed to detect and counteract adversarial modifications, thus enhancing model robustness. **METAATTACK** (Zügner and Günnemann 2019) performed both poison and evasion attacks on GNNs by treating the graph structure as a hyperparameter that can be optimized. The attack directly modifies the graph structure using gradient descent, formulating the problem as a bi-level optimization task. Their method utilized meta-gradients, which capture how small perturbations in the graph structure affect the attacker’s loss after training, enabling precise updates to the graph that degrade the model’s performance.

For the transfer study underlying Contribution 1, we adopt **NETTACK** (Zügner, Akbarnejad, and Günnemann 2018) and **METAATTACK** (Zügner and Günnemann 2019) as representative adversarial attacks spanning both strategies: poisoning attacks target model integrity during training, while evasion attacks compromise performance at inference time.

A.3 How NETTACK and METAATTACK Work

NETTACK performs graph structure perturbation by adding and removing edges between nodes, while ensuring that the degree of the perturbed graph is preserved. It uses a greedy optimization technique to compute the minimal perturbations that will maximize the model’s loss function during training, with respect to a surrogate model. This surrogate model is a linearized 2-layer graph convolutional network (GCN), which approximates how changes in the graph structure affect the

classification result. The primary objective of NETTACK is to increase the model’s confidence in incorrect classifications by maximizing the distance between the logits of the perturbed graph and those of the clean graph. To achieve this, NETTACK identifies candidate edges for perturbation around a target node and evaluates a score function, for each connected edge. This score indicates how significantly altering a particular edge will impact the classification result of the target node. The edge with the highest score is then selected, and the graph is updated by either adding or removing that edge. Since NETTACK is a targeted attack (only cause misclassification of a specific single node), we adapt the same method in (Zügner and Günnemann 2019) to convert it into untargeted attack (compromise node classification performance of the model) which randomly select one test node and attack it using NETTACK while considering all nodes in the network. We then aggregate the result over the test nodes.

METAATTACK optimizes the graph structure directly via gradient descent by treating the adjacency matrix (graph structure) as a hyperparameter. It frames the attack as a bi-level optimization problem: the inner level minimizes the training loss on the original graph, while the outer level minimizes the loss on the poisoned graph. To achieve this, METAATTACK utilizes a surrogate model, specifically a linearized two-layer GCN, to approximate how changes in the graph structure affect model performance. A score function evaluates the impact of each perturbation, and the attack flips the sign of meta-gradients (gradient wrt hyperparameters) for connected nodes, indicating which edges to remove. The algorithm then greedily selects perturbations—either adding or removing edge based on those with the highest scores, to maximize the attack’s impact.

For both methods, we use the same unnoticeability constraint introduced by (Zügner, Akbarnejad, and Günnemann 2018), which ensures that the graph’s degree distribution only changes slightly. Therefore, we only perturb 10% of the edges.

In our poisoning attack experiments with LLAGA, we employ the perturbed graph structure obtained from NETTACK or METAATTACK to generate the node-level sequences and retrain the projector that converts the node-level sequences into a sequence of token embeddings. Similarly, for GRAPH-PROMPTER, we utilize the perturbed graph structure, re-encode the graph using a graph attention network (GAT), and retrain the projector with the perturbed graph. For evasion attacks, the graph-aware LLM is trained on unperturbed data, with perturbations introduced only during the inference stage.

A.4 Algorithm for Node Sequence Template Injection Attacks

Algorithm 1 formalizes the node sequence template injection attack (Contribution 3). The model constructs a *fixed-shape* computational tree for each node u , sampling k neighbors per node up to depth d (lines 1–14); when a node has fewer than k neighbors, placeholders fill the remaining slots, and this is precisely the attack surface we exploit. By manipulating the graph, an attacker injects malicious nodes into those placeholder positions. Our three strategies escalate in

Algorithm 1: Injection Attacks on Neighborhood Detail Template for Obtaining Node Sequence.

Input: Graph $G = (V, E)$, target node u , tree depth d , children size k

Output: Modified graph G' with injected nodes

```

1: procedure NEIGHBORHOODDETAILTEMPLATE( $u, d, k$ )
2:    $T \leftarrow \emptyset$  ▷ Initialize empty tree
3:    $T.\text{root} \leftarrow u$ 
4:   for level  $\ell = 1$  to  $d$  do
5:     for each node  $v$  at level  $\ell - 1$  do
6:        $N_v \leftarrow \text{Sample}(\mathcal{N}(v), k)$  ▷ Sample  $k$  neighbors
7:       if  $|N_v| < k$  then
8:         Fill remaining slots with placeholders
9:       end if
10:      Add  $N_v$  as children of  $v$  in  $T$ 
11:    end for
12:  end for
13:  return  $T$ 
14: end procedure

15: procedure ATTACK( $G, u, d, k, \text{AttackStrategy}$ )
16:    $G' \leftarrow G$  ▷ Create copy of original graph
17:    $T \leftarrow \text{NEIGHBORHOODDETAILTEMPLATE}(u, d, k)$ 
18:    $P \leftarrow \text{GetPlaceholders}(T)$ 
19:   if  $|P| > 0$  then
20:      $\mathcal{N}_2 \leftarrow \text{2-hop neighborhood of } u$ 
21:     if AttackStrategy = NI then
22:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
23:        $V_{\text{inject}} \leftarrow \text{Sample}(V_{\text{non-adjacent}}, |P|)$ 
24:       for each placeholder position  $p \in P$  do
25:          $v \leftarrow \text{Next node from } V_{\text{inject}}$ 
26:         Add edge  $(\text{parent}(p), v)$  to  $G'$ 
27:       end for
28:     else if AttackStrategy = SI then
29:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
30:        $s \leftarrow \text{GetHighestDegreeNode}(V_{\text{non-adjacent}})$ 
31:       for first placeholder position  $p \in P$  do
32:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
33:         break ▷ Only use first placeholder
34:       end for
35:     else if AttackStrategy = MSI then
36:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
37:        $S \leftarrow \text{GetNodesOrderedByDegree}(V_{\text{non-adjacent}})$ 
38:       for  $i = 1$  to  $|P|$  do
39:          $s \leftarrow \text{Get } i\text{-th node from } S$ 
40:          $p \leftarrow \text{Get } i\text{-th placeholder from } P$ 
41:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
42:       end for
43:     end if
44:   end if
45:   return  $G'$ 
46: end procedure

```

strength: *Non-Adjacent Injection (NI)* inserts nodes outside the 2-hop neighborhood of u (lines 21–27); *Supernode Injection (SI)* injects a single high-degree node into one place-

holder (lines 29–33); and *Multiple Supernode Injection (MSI)* distributes distinct high-degree nodes across all placeholders (lines 35–43). In each case, inserting uninformative nodes corrupts the target node’s representation in the sequence and thereby impairs training of the projection layer.

A.5 Ablation Studies

Here, we present the results for the ablation studies presented in the main paper.

Ablation Results: Varying Feature Perturbation Figure 6 reports how LLGA and GRAPHPROMPTER behave as the percentage of perturbed features varies. Increasing the proportion of perturbed features consistently degrades accuracy for both models, reinforcing the feature-level vulnerability that is central to Contribution 4.

Ablation Results: Does the Choice of LLM Affect Attack Performance? The results in Table 8 show that the choice of base LLM has minimal impact on both clean performance and adversarial vulnerability in the LLGA framework. Across Cora, Citeseer, and PubMed, models built on Vicuna-7B and Llama2-7B achieve nearly identical accuracy under clean conditions and exhibit similar degradation under both poisoning and evasion attacks. For instance, under METAATTACK evasion, Cora performance drops to 0.44 ± 0.06 for Vicuna-7B and 0.43 ± 0.08 for Llama2-7B. These differences are well within standard deviation. Similar patterns hold across all datasets and attack types. This indicates that attack effectiveness is largely independent of the base LLM.

A.6 Results on the ArXiv Dataset

To test whether our new attack surface (Contribution 3) scales, we evaluate it on the large-scale ArXiv dataset (169,343 nodes, 1,166,243 edges, sparsity 0.81). Table 9 reveals a striking reversal: on ArXiv, poisoning consistently outperforms evasion across all three injection strategies – the opposite of what we observe on Cora, Citeseer, and PubMed. This suggests that ArXiv has structural characteristics that make it more vulnerable to poisoning yet more robust to evasion, and that even at very low sparsity, inserting a single supernode can meaningfully degrade performance.

A.7 Feature Attack Examples and Defense Template

Feature Attack Examples The examples in the prompt, as shown in Appendix A.7, illustrate our adversarial attacks on real features from the Cora dataset. The “input” is the text received by the LLM where the attack is being applied. It is important to note that while a human reader perceives the original text and the perturbed text as visually identical or very similar, the LLM processes a sequence of different Unicode characters or a different character order, which can alter its behavior. We show this in Table 10 for homograph and Table 11 for reordering attack.

Feature Corrector Defense Template We designed a feature corrector module to mitigate the effects of imperceptible attacks, specifically homograph substitutions and character

reorderings. This module is implemented as a prompt-based defense using LLMs. The prompt is carefully crafted to instruct the LLM to act as a linguistic validator, focusing on detecting and correcting subtle, human-imperceptible perturbations.

The template for our prompt is shown below. It explicitly states the task and provides a clear output format, which significantly improves the LLM’s performance.

A.8 Detailed Discussion

This section expands on the four observations that support our contributions. As graph-structured data is increasingly integrated into LLMs, it is essential to weigh not only their performance benefits but also their vulnerabilities. Our systematic investigation of two representative graph-aware LLMs, LLGA and GRAPHPROMPTER, exposes significant weaknesses under the poisoning and evasion attacks traditionally aimed at GNNs. We discuss the key observations below.

Impact of Graph Encoding on Adversarial Vulnerability The central takeaway – and the evidence behind our encoding-based taxonomy (Contribution 2) – is that how graph structure is integrated into an LLM can inadvertently increase its susceptibility to adversarial attacks. Specifically, the node sequence template in LLGA opens a vulnerability that adversaries can exploit to substantially degrade performance (Contribution 3). This LLGA-specific attack surface exposes a critical design tradeoff: while node-level templates are an effective way to incorporate graph structure and yield strong task performance, they also introduce attack vectors absent from traditional GNNs. GRAPHPROMPTER, which encodes graphs with a GNN-based projector, is comparatively more resilient, pointing to the promise of hybrid architectures that combine the strengths of LLMs and GNNs.

Sensitivity of Graph-aware LLMs to Feature Perturbations Beyond the encoding vulnerability, our results establish the effectiveness of feature perturbation attacks (Contribution 4). Despite their different structural encodings, both LLGA and GRAPHPROMPTER are highly sensitive to imperceptible perturbations of the node features. This reflects a broader trend: although these models excel at handling text, their robustness to adversarial perturbations of that same text remains a critical weakness. That an adversary can degrade performance by subtly and undetectably altering node attributes calls the integrity of graph-aware LLMs into question, especially in security-sensitive applications. Text sanitization techniques such as those of (Boucher et al. 2022) offer partial defenses, but countering imperceptible manipulations like homograph substitution remains hard – which is exactly why we use an LLM-based corrector (Contribution 5) to detect and repair suspicious inputs before downstream processing.

Data-dependent Variability of Poisoning and Evasion Attacks on Graph-aware LLMs Our findings show that the efficacy of poisoning and evasion attacks is not uniform across datasets. Attacks are particularly effective on Cora, Citeseer, and PubMed, whereas ArXiv exhibits a more nuanced interaction between attack strategy and graph structure. This variability underscores that vulnerability assessments

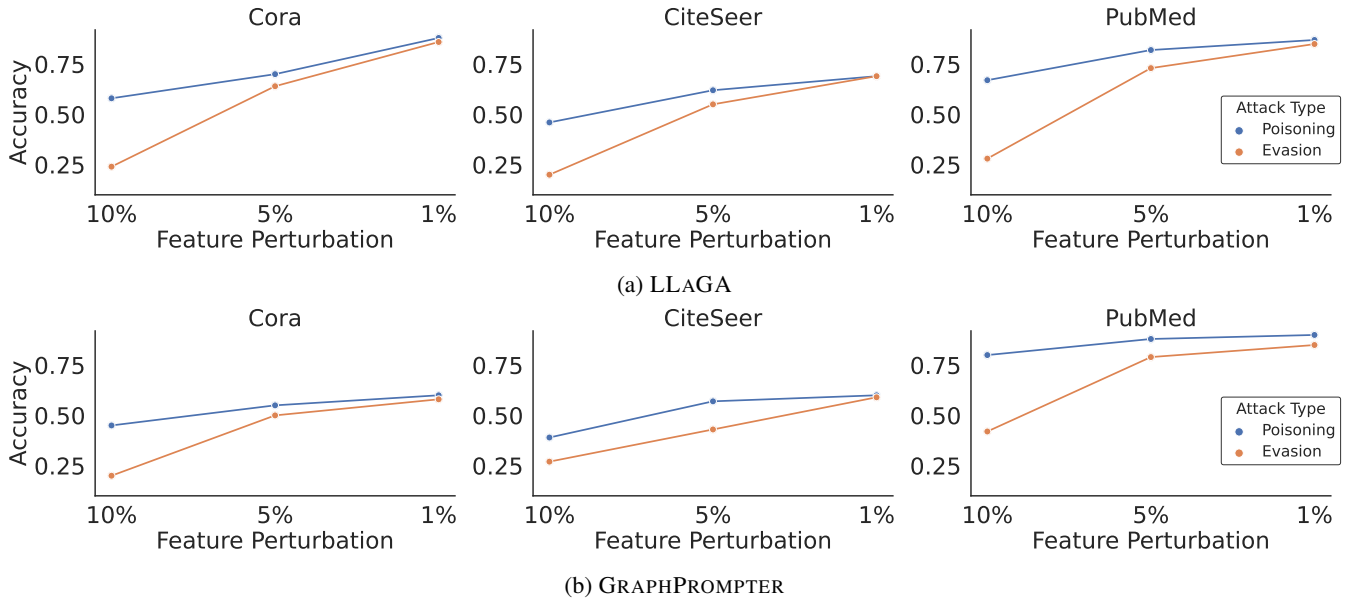


Figure 6: Performance of the Graph-aware LLM when the percentage of the perturbed feature is varied. The lower the accuracy, the better.

Table 8: Impact of base model changes on model’s performance and under different attacks. We show the result for LLAGA. The results demonstrate that the choice of base model does not significantly affect susceptibility to attacks.

Data	Base Model	Clean	Poisoning			Evasion		
			NETTACK	METAATTACK	MSI	NETTACK	METAATTACK	MSI
Cora	Vicuna-7B	0.89 ± 0.07	0.87 ± 0.04	0.79 ± 0.03	0.82 ± 0.04	0.55 ± 0.09	0.44 ± 0.06	0.30 ± 0.05
	Llama2-7B	0.90 ± 0.03	0.86 ± 0.02	0.77 ± 0.05	0.82 ± 0.02	0.56 ± 0.07	0.43 ± 0.08	0.31 ± 0.01
Citeseer	Vicuna-7B	0.71 ± 0.04	0.64 ± 0.05	0.63 ± 0.04	0.62 ± 0.02	0.59 ± 0.04	0.50 ± 0.03	0.24 ± 0.05
	Llama2-7B	0.71 ± 0.05	0.65 ± 0.03	0.60 ± 0.03	0.61 ± 0.04	0.58 ± 0.03	0.48 ± 0.05	0.26 ± 0.03
PubMed	Vicuna-7B	0.90 ± 0.04	0.89 ± 0.03	0.89 ± 0.08	0.81 ± 0.03	0.84 ± 0.05	0.73 ± 0.04	0.76 ± 0.03
	Llama2-7B	0.91 ± 0.06	0.88 ± 0.04	0.90 ± 0.02	0.80 ± 0.04	0.85 ± 0.03	0.75 ± 0.02	0.77 ± 0.02

must account for dataset characteristics, not just algorithmic design – an important caveat as these models are deployed across diverse real-world domains.

Implications for Graph-aware LLM Development Taken together, the vulnerabilities exposed here carry broader implications for how graph-aware LLMs should be built and deployed. As they increasingly take over tasks once handled by GNNs, designers must explicitly weigh performance against these newly exposed risks. Our results point to a pressing need for more robust encoding techniques and for defenses against adversarial manipulation, particularly feature perturbations. Finally, by clarifying how adversarial attacks transfer from GNNs to graph-aware LLMs (Contribution 1), our work opens new avenues for protecting these models against sophisticated adversaries.

Difficulty in Applying Existing GNN defenses Against Adversarial Attacks on Graph-aware LLMs Defenses on adversarial attacks based on feature perturbation. This subsection explains why Contribution 4 forces a new defense

and directly motivates Contribution 5. Adversarial attacks on GNNs have focused on structural perturbations, since feature-based perturbations typically fail to change the predicted class of a target node (Dai et al. 2018; Wu et al. 2019; Zügner and Günnemann 2019). As a result, there are *no known defenses against feature-based attacks on GNNs*. Our main-paper results overturn this assumption: feature perturbations are highly effective in the graph-aware LLM paradigm, so attacks that were harmless against GNNs can now manipulate model behavior. This shift demands defenses that address both structural and feature-based perturbations. Text sanitization such as OCR-based preprocessing offers only a partial remedy, because OCR is imperfect and misreads homoglyphs at a higher rate than unperturbed text (Boucher et al. 2022). This gap is precisely what led us to design an LLM-based self-correcting mechanism for feature perturbations.

Defenses on adversarial attacks based on structural per-

Prompt for Feature Corrector

You are a linguistic validator. Your task is to detect and correct any subtle, imperceptible text perturbations such as homoglyph substitutions (e.g., replacing "o" with "0") and character reorderings. These changes are often used to attack machine learning models. Given a text string, carefully analyze it. If you identify any perturbations, provide the corrected, original-looking version of the text. If no perturbations are found, return the original text unchanged.

Example 1 (Homoglyph Substitution):

Input: Title: The mgprir heuristic fr discvring prtin squnc pttrens Abstract: Svrl cmputr lgrithms fr discvring pttrens in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rlted squncs.

Output: Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Example 2 (Character Reordering):

Input: Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dscribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiottn of the apclpiation of three machine larening algorithms-lR, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.

Output: Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-lR, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Example 3 (Mixed Homoglyph & Reordering):

Input: Title: Submitted to NIPS96, Sctin: Applications. Prfrnc: Oral prsentatin Reinforcement Lrning for Dynmic Channl Allocation in Abstract: In ellulr tlphne systms, n imrtnt prblm is to dynmiclly alloct th cmmunictin rsurc (chnnls) so as to mximize srvc in stchstic cllr nvirnmt. This prblem is naturally frmultd s dynmic prgrmming prblm and we us a reinforcement lerning (RL) mthd to find dynmic chnnl llection plicies that ar bttr than prvius heuristic slutins.

Output: Title: Submitted to NIPS96, Section: Applications. Preference: Oral presentation Reinforcement Learning for Dynamic Channel Allocation in Abstract: In cellular telephone systems, an important problem is to dynamically allocate the communication resource (channels) so as to maximize service in a stochastic caller environment. This problem is naturally formulated as a dynamic programming problem and we use a reinforcement learning (RL) method to find dynamic channel allocation policies that are better than previous heuristic solutions.

Your Task: Text to correct: {perturbed_text_here}

Corrected Text:

Figure 7: The prompt template used for the LLM-based feature corrector module. It explicitly defines the role, task, and expected output format for the LLM.

Table 9: Node sequence template injection attack (Contribution 3) extended to the large-scale ArXiv dataset alongside Cora, Citeseer, and PubMed. *NI*, *SI*, and *MSI* denote the non-adjacent, single-supernode, and multiple-supernode injection attacks; Clean is the unattacked model. Note the poisoning/evasion reversal on ArXiv relative to the smaller datasets.

Data	Attack	NI	SI	MSI
Cora	Clean	————	0.89 $\pm$ 0.07	————
	Poisoning	0.86 $\pm$ 0.03	0.84 $\pm$ 0.06	0.82 $\pm$ 0.04
	Evasion	0.42 $\pm$ 0.08	0.36 $\pm$ 0.05	0.30 $\pm$ 0.05
Citeseer	Clean	————	0.71 $\pm$ 0.04	————
	Poisoning	0.69 $\pm$ 0.05	0.66 $\pm$ 0.03	0.62 $\pm$ 0.02
	Evasion	0.36 $\pm$ 0.06	0.30 $\pm$ 0.04	0.24 $\pm$ 0.05
PubMed	Clean	————	0.90 $\pm$ 0.04	————
	Poisoning	0.88 $\pm$ 0.04	0.85 $\pm$ 0.05	0.81 $\pm$ 0.03
	Evasion	0.85 $\pm$ 0.07	0.82 $\pm$ 0.04	0.76 $\pm$ 0.03
ArXiv	Clean	————	0.73 $\pm$ 0.02	————
	Poisoning	0.50 $\pm$ 0.03	0.43 $\pm$ 0.04	0.42 $\pm$ 0.03
	Evasion	0.71 $\pm$ 0.06	0.69 $\pm$ 0.02	0.60 $\pm$ 0.03

turbation. Because structure-based attacks transfer from GNNs to graph-aware LLMs (Contribution 1), especially under evasion, one might expect GNN defenses to carry over. They do not. Unlike the traditional GNN setting, *graph-aware LLMs are also susceptible to feature perturbations, which renders these structural defenses ineffective.*

Consider a representative GNN defense that makes the adjacency matrix trainable so edge weights adapt during training, assigning lower weights to edges between dissimilar nodes and higher weights to edges between feature-similar nodes. Several **preprocessing-based** defenses build on this idea, e.g., removing edges between low-similarity nodes (Wu et al. 2019) or randomized edge dropping (Dai et al. 2018). All of them hinge on clean, unperturbed node features to compute reliable similarity scores. When features are perturbed too – as in our attacks – these defenses not only fail but can backfire by wiring up incorrect edges. Figure 8 makes this concrete on a random Cora node: the clean feature-similarity graph preserves some original edges (while adding a few spurious ones), whereas the graph reconstructed from perturbed features preserves no original connections at all, producing entirely wrong reconstructions.

A second line of work decouples reliance on the graph structure. (Shanthamallu, Thiagarajan, and Spanias 2021) trains a surrogate predictor on node features alone and uses uncertainty matching to extract graph information from the GNN, while Wu et al. (2021) co-trains submodels independently on feature and structural views, distilling knowledge by adding confident unlabeled data to the training set. Yet these too assume unperturbed node features – an assumption our setting violates, since we perturb both structure and features. The conclusion is clear and motivates Contribution 5: GNN structural defenses do not transfer to graph-aware LLMs, and this paradigm needs defenses purpose-built for it.

Table 10: An illustration of the imperceptible homoglyph attack. While the perturbed text is processed as distinct characters by the LLM, the visual output is nearly identical for a human, as shown in the image.

Original Feature	What LLM Sees	What a Human Sees
Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.	Title: The mgprir huristic fr discvring prtln squnc pttrns Abstract: Svrl cmputr lgrithms fr discvring pttrns in grups f prtln squncs r in us tht r bsd on fitting th prmtrs f stttisticl mdl to grup of rltd sauncs.	Title: The megaprior heuristic for discovering protein sequence patterns. Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Table 11: An illustration of the imperceptible reordering attack. While the perturbed text is processed as distinct characters by the LLM, the visual output is nearly identical for a human, as shown in the image.

Original Feature	What LLM Sees	What a Human Sees
Title: Applications of machine learning: a medical follow up study. Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.	Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dsceribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiotn of the aplclpiation of three machine larening algorithms-1R, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.	Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

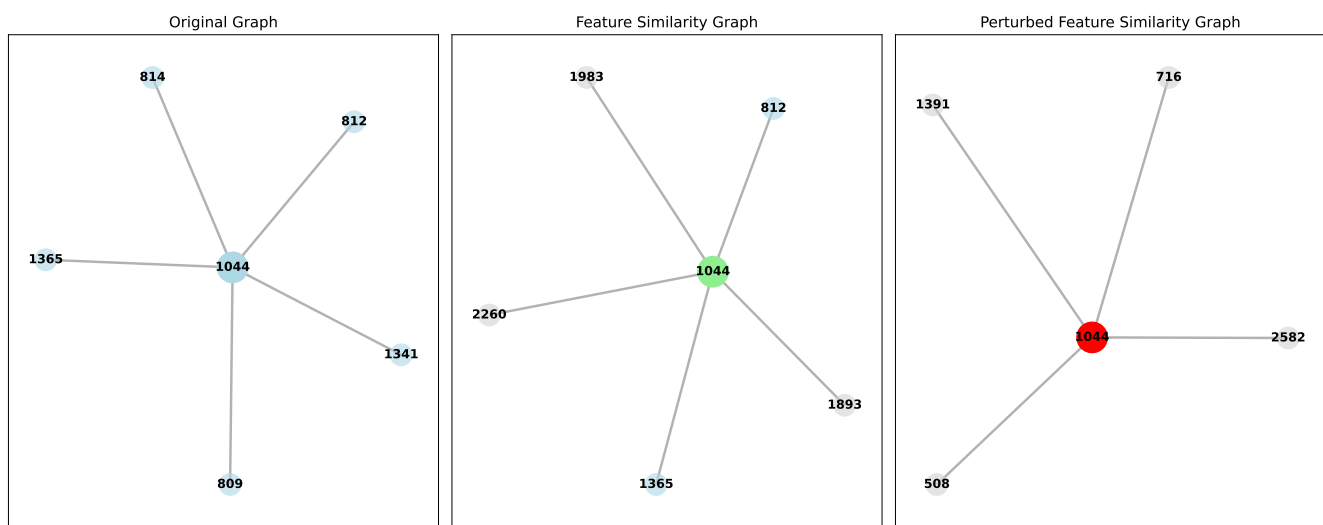


Figure 8: Edges reconstructed for a random node 1044. **Original (left)** is the ground truth edges from the original graph. **Feature Similarity (middle)** shows the edges reconstructed using clean node features, where blue nodes indicate preserved connections from the original graph. **Perturbed Feature Similarity (right)** shows the edges reconstructed using perturbed node features.